Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 8

Course: Operating Systems

Topic: The Same Calls, Inside Your Own OS (xv6)

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – RUN, INSIDE XV6 **2**

1.1 Observe 3

2 PART 2 – PINGPONG **4**

2.1 Observe 4

2.2 Optional: primes 5

Conclusions **6**

PURPOSE OF THE WORK

The purpose of this laboratory work is to run inside xv6, the small teaching operating system, the same calls that were run on Linux in Lab 7: `fork`, `exec` and `wait`. In Part 1 the program `run.c` is written for xv6, where there is no glibc and the program uses only the system calls of the kernel that is compiled together with it, and it is changed twice to see a successful and a failed `exec`. In Part 2 the program `pingpong.c` makes two processes exchange one byte through two pipes. The optional program `primes.c` (a sieve of processes) was also written. The work was done on the current xv6-riscv tree (commit `f780d78`), built with `riscv64-unknown-elf-gcc` and run in QEMU 8.2 on Ubuntu 24.04 (a Debian-family Linux). Every output below was produced by these programs in xv6, with `make qemu`, on a fresh disk image. The answers to all *Observe* boxes are given in the corresponding parts.

```
UPROGS=\
    $U/_cat\
    ...
    $U/_sync\
    $U/_run\
    $U/_pingpong\
    $U/_primes\
```

Figure 0.1 – The new lines of the Makefile (each starts with a TAB)

1 PART 1 – RUN, INSIDE XV6

The file `user/run.c` is the code of the assignment, without changes. It does the same three steps as `run.c` of Lab 7: `fork`, `exec` (the child becomes `echo`) and `wait`.

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                 // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                     // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);             // step 2: become echo
    printf("exec failed\n");        // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                    // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Figure 1.1 – user/run.c

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Figure 1.2 – make qemu, then run

Then the two lines of the child were changed, as asked in 1.3, so that it runs `ls` (the listing is shortened: the dots stand for the programs between `README` and `sync`; `run`, `pingpong` and `primes` are there because the programs of this work are on the disk too):

```
char *args[] = { "ls", 0 };
exec("ls", args);              // step 2: become ls

$ run
parent: my pid is 3
child: my pid is 4
.               1 1 1024
..              1 1 1024
README          2 2 2441
...
sync            2 22 35040
run             2 23 35944
pingpong        2 24 35960
primes          2 25 36624
console         3 26 0
parent: child 4 ended with status 0
```

Figure 1.3 – The child runs ls

Last, in 1.4, a program that does not exist was executed:

```
exec("nosuchprog", args);      // step 2: become nosuchprog

$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

Figure 1.4 – exec of a program that does not exist

1.1 Observe

1. **The three calls, in xv6 and on Linux.**

Step	xv6 (this lab)	Linux (Lab 7)	Meaning
1	`fork()`	`fork()`	copy the process
2	`exec("echo", args)`	`execlp("ls", "ls", "-l", NULL)`	become another program
3	`wait(&status)`	`waitpid(pid, &status, 0)`	wait for the child

   What is the same: the three steps and their order, and their meaning. `fork` returns 0 in the child and the PID of the child in the parent. A successful `exec` replaces the program of the process and does not return, so the line after it runs only on a failure. `wait` blocks the parent until the child has ended and gives its status. The shell of both systems does the same for every command. The differences are small: `execlp` of glibc searches the program in `PATH` and takes the arguments as a list, while the `exec` of xv6 is the bare system call (the name of the file and an array of arguments that ends with 0, and no `PATH`); and `wait` of xv6 gives the exit code itself, while on Linux the macro `WEXITSTATUS` was used to take it out of the status word.

2. **Who prints "hello from the child"?** The program `echo`. The code of `run` only printed "child: my pid is 4" and then called `exec`, which replaced the whole program of the child with `echo` and gave it the arguments `hello from the child`. `echo` writes its arguments to the descriptor 1, and the old code of `run` does not exist any more in the child (in the same way `ls` printed the listing in Figure 1.3).

3. **Why the status is 1 here and 127 on Linux.** Only because of the number written in the source after the failed `exec`: the `run.c` of xv6 has `exit(1)`, and the one of Lab 7 has `exit(127)`. The child ends with this number, and the parent just reports what it receives (Figure 1.4). The value 127 is the convention of the Linux shell for "command not found"; xv6 has no such convention in this program, so the usual code of an error, 1, was used. With a working program the status is 0 in both systems.

2 PART 2 – PINGPONG

The parent sends one byte to the child through the first pipe (`p2c`). The child prints its own PID and "received ping", writes the byte back through the second pipe (`c2p`) and exits. The parent reads it, prints "received pong" and waits for the child. The TODO lines of the template were completed with `read` and `write` of one byte; the variable `b` was given the value `'x'` at the start, so that a defined byte is sent.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];   // parent-to-child, child-to-parent
  char b = 'x';         // the byte that travels there and back

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);  // the ends the child does not use
    read(p2c[0], &b, 1);           // wait for the ping
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);          // send the pong
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);    // the ends the parent does not use
  write(p2c[1], &b, 1);            // send the ping
  read(c2p[0], &b, 1);             // wait for the pong
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

Figure 2.1 – user/pingpong.c

```
$ pingpong
4: received ping
3: received pong
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
```

Figure 2.2 – pingpong, run three times

In every run "ping" comes first, from the child (the larger PID), and "pong" comes second, from the parent; the PIDs change from run to run.

2.1 Observe

1. **Why two pipes.** A pipe is a one-way channel: the bytes written at the writing end are read at the reading end, and each byte is read once. With one pipe for both directions the two processes would write and read in the *same* buffer, and nothing says who gets which byte: after the parent writes the ping, it reads at once and can take

its own ping back before the child has read it, so the child would wait for a byte that is already gone. Two pipes give two separate directions, and each process reads only what the other one wrote. For the same reason each process closes the ends it does not use (`close(p2c[1]); close(c2p[0]);` in the child and the other two in the parent): the descriptors are not wasted, and a reader can get the end of file when the writer ends.

2. **Which call makes the parent wait.** The `read` on the pipe, `read(c2p[0], &b, 1)`. When the pipe is empty and a writing end is still open, `read` does not return: the kernel puts the process to sleep until the child writes a byte. So the parent cannot print "pong" before the child has replied, and the child in its turn is held by `read(p2c[0], ...)` until the ping comes. This is the reason why "ping" always comes first. The call `wait(0)` at the end only waits for the *end* of the child (it collects it), not for the answer.

2.2 Optional: primes

The program `user/primes.c` prints the primes from 2 to 35 with a chain of processes (the sieve of Eratosthenes made of pipes). The first process writes the numbers 2, 3, ..., 35 into a pipe. Each stage reads its first number `p`, which is a prime (nothing smaller divided it), prints `prime p`, creates a pipe and a child, and sends to the child only the numbers that `p` does not divide. Every stage closes the pipe ends that it does not use, and when the input ends it closes its output, so the next stage also reaches the end of file and the whole chain ends.

```
#include "kernel/types.h"
#include "user/user.h"

// One stage of the sieve: rd is the pipe with the numbers from the left.
void sieve(int rd) __attribute__((noreturn));

void
sieve(int rd)
{
  int p, n;

  if(read(rd, &p, sizeof(p)) != sizeof(p)){   // nothing left: the chain ends
    close(rd);
    exit(0);
  }
  printf("prime %d\n", p);                    // the first number is a prime

  int pp[2];
  pipe(pp);
  if(fork() == 0){
    close(pp[1]);                             // the next stage only reads
    close(rd);
    sieve(pp[0]);
  }
  close(pp[0]);                               // this stage only writes
  while(read(rd, &n, sizeof(n)) == sizeof(n)){
    if(n % p != 0)
      write(pp[1], &n, sizeof(n));            // forward what p does not divide
  }
  close(rd);
  close(pp[1]);                               // the next stage sees end of file
  wait(0);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int fd[2];
```

```
  pipe(fd);
  if(fork() == 0){
    close(fd[1]);
    sieve(fd[0]);
  }
  close(fd[0]);
  for(int i = 2; i <= 35; i++)
    write(fd[1], &i, sizeof(i));
  close(fd[1]);
  wait(0);
  exit(0);
}
```

Figure 2.3 – user/primes.c

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

Figure 2.4 – primes

These are exactly the 11 primes up to 35. The program finished without errors, which confirms that the ends were closed correctly (a forgotten `close` would keep the chain waiting or use up the 16 descriptors of a process).

CONCLUSIONS

In this laboratory work the calls of Lab 7 were run inside xv6 and the result was the same: `fork` copies the process, `exec` replaces its program and returns only on a failure, and `wait` gives the status of the child to the parent. The differences were only in the library around the calls (no `PATH` search, no status macros) and in the exit code chosen by the programmer after a failed `exec`: 1 in xv6 and 127 in the Linux program. The program `echo` printed the message, not the code of `run`, because `exec` had replaced it.

The program `pingpong` showed how two processes talk: each pipe works in one direction only, so two pipes are needed for a question and an answer, and the blocking `read` is what makes the parent wait for the child and fixes the order “ping”, then “pong”. The optional program `primes` used the same tools in a chain of processes and printed the 11 primes up to 35, and it showed again that every unused end of a pipe must be closed.